

HACETTEPE ÜNİVERSİTESİ
YAZILIM MÜHENDİSLİĞİ YÜKSEK LİSANSI

Nesneye Yönelik Yazılım Geliştirme

Ara Sınav · Cevap Belgesi

Nurcan Denli Bayır

Öğrenci No: N25110987

nurcandenli25@hacettepe.edu.tr

DÖNEM 2025-2026 Bahar

TESLİM TARİHİ 30 Nisan 2026

Bu belge, sınavda istenen iki senaryo için nesneye yönelik tasarım çözümlerini sunar. Her soru için (i) senaryo çözümlemesi, (ii) tasarım örüntüsü seçimleri ile *reddedilmiş alternatiflerin gerekçeleri*, (iii) UML 2.x sınıf diyagramı, (iv) bir veya iki sıra (sequence) diyagramı ve (v) tipik çalışma akışları yer almaktadır. Diyagramlar, bağlantılık (coupling) ve uyum (cohesion) ölçütleri ile *okunaklılık* dengesi gözetilerek elle dizayn edilmiş; otomatik olarak yerleştirilmemiştir.

İÇİNDEKİLER

Soru 1 · Akıllı Ev Yönetim Sistemi *State + Mediator + Observer*

Soru 2 · SimpleCar E-Satış Sistemi - *Strategy + CoR + Command + Memento (+ Abstract Factory)*

SORU 1 · 50 PUAN

Akıllı Ev Yönetim Sistemi

State, Mediator ve Observer Örüntülerinin Birlikte Kullanımı

1.1 Senaryonun Çözümlemesi

Senaryo, dört tip bileşen tanımlar: iki *aktif sistem* (aydınlatma, ısıtma), iki *pasif ölçüm kaynağı* (kapı sensörü, hareket sensörü) ve bir *orta kontrol noktası* (akıllı ev izleme motoru). Bileşenler arasındaki etkileşimleri dört temel olguya indirgeyebiliriz:

- Davranışsal çoğulluk.** Aydınlatma ve ısıtma sistemleri, iç durumlarına — yani aktif *moda* — göre *farklı davranır*. Tasarruflu modda ısıtma 18°C, aydınlatma asgari seviye hedefler; güvenli modda ise sırasıyla 40°C panel sıcaklığı ve maksimum aydınlatma uygulanır. Bu davranışsal çoğulluğu **if/switch** ile çözmek, her yeni mod eklendiğinde mevcut sınıfı değiştirmeyi gerektirir. Polimorfizm üzerinden taşınması *açık-kapalı* prensibinin (OCP) doğal sonucudur.
- Koşullu senkronizasyon.** İki sistemin modu *bağlantılıdır*: bir sistemde tasarruflu mod seçilirse diğeri de zorunlu olarak geçer. Bu kural sistemler arasında *doğrudan* uygulanırsa, her sistem diğeri referans alır; $n \times (n-1)$ bağlantısı ortaya çıkar. Senaryoda iki sistem var, ancak ileride yeni bir sistem (örn. sulama) eklendiğinde doğrudan referans yapısı kırılğan hale gelir.
- Olay yayını, alıcı farkındalığı olmadan.** Sensörler, ölçümlerini iletmeli; ama *kime* ilettiklerini *bilmek zorunda olmamalı*. Sensörün motora doğrudan referansı olması, sensörü izleme altyapısına bağlar — sensörü başka bir bağlamda (örn. test, ayrı bir konsol) kullanmayı imkânsızlaştırır.
- Çoğul bildirim hedefi.** Bir kullanıcının *birden fazla* istemcisi olabilir (mobil ve web). Motor, "kullanıcıya bildirim gönder" eylemini somut bir istemci tipine bağlarsa, yeni bir kanal (e-posta, SMS, sesli asistan) eklendiğinde motor değişir. Burada da *yayıncı-abone* ayrışması kaçınılmazdır.

1.2 Örüntü Seçimleri ve Gerekçeleri

Yukarıdaki dört olgu, GoF kataloğundaki üç *davranışsal* örüntü ile noktasal olarak eşleşir:

ÖRÜNTÜ	UYGULANAN YER	ÇÖZDÜĞÜ OLGU
State <i>GoF, s.305</i>	<code>CalismaModu</code> arayüzü; <code>GüvenliMod</code> ve <code>TasarrufluMod</code> ; <code>EvSistemi</code> «bağlam» rolünde, <code>aktifMod</code> alanını tutar.	(1) Davranışsal çoğulluk: <code>komutCalistir</code> çağırıldığında davranış aktif moda göre polimorfik seçilir. Yeni bir mod eklemek <code>EvSistemi</code> 'ni değiştirmez.

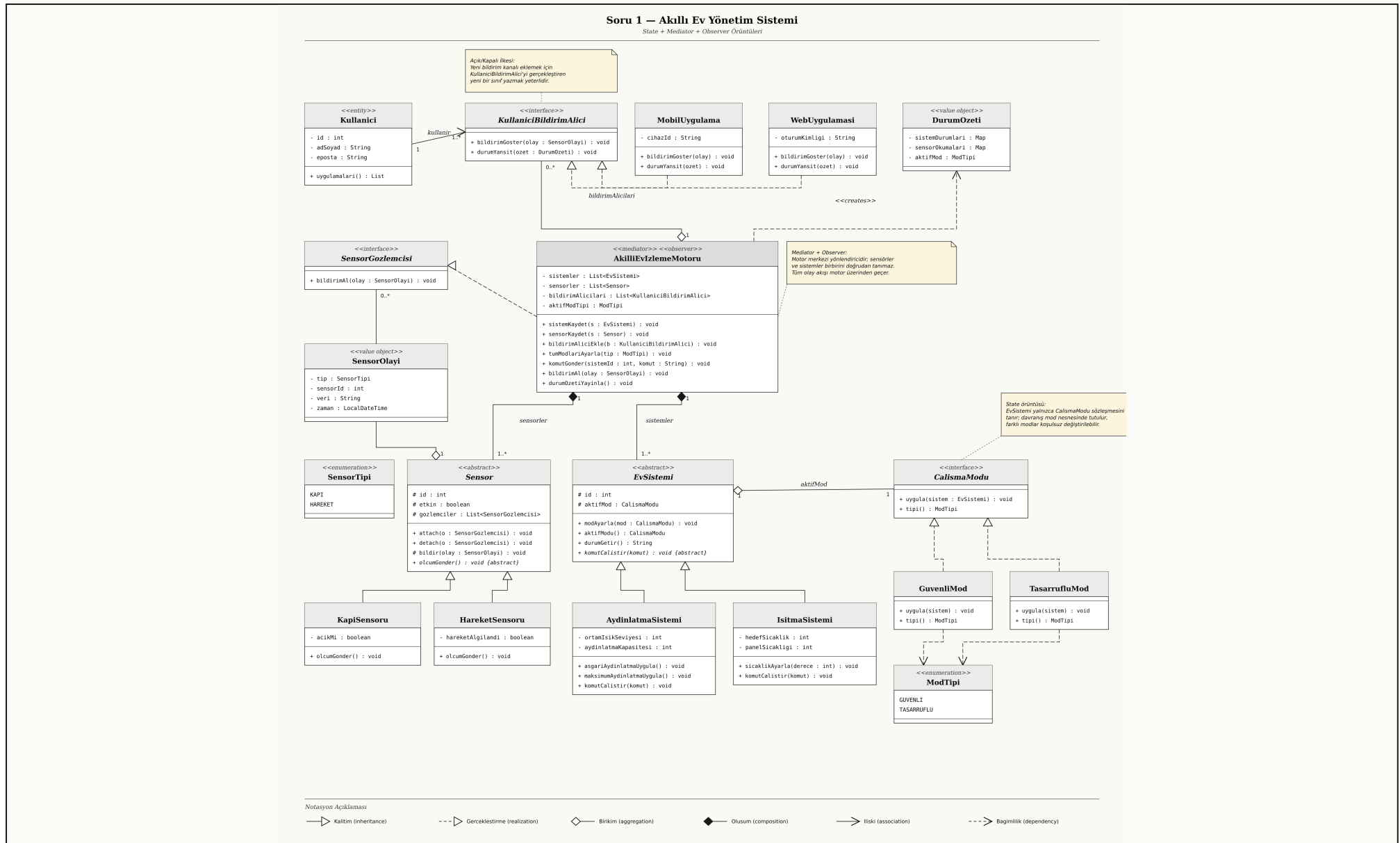
ÖRÜNTÜ	UYGULANAN YER	ÇÖZDÜĞÜ OLGU
Mediator <i>GoF, s.273</i>	AkıllıEvIzlemeMotoru ; tüm EvSistemi ve Sensor nesnelerini tanır, sistemleri birbirine bağlamadan koordine eder.	(2) Koşullu senkronizasyon: "tüm sistemler aynı moda geçsin" gibi çok-yönlü kurallar arabulucuda <i>tek bir noktada</i> uygulanır; sistemler arası referans gerekmez.
Observer <i>GoF, s.293</i>	(a) Sensor ↔ SensorGozlemcisi arayüzü (motor abonedir). (b) AkıllıEvIzlemeMotoru ↔ KullaniciBildirimAlici arayüzü (mobil/web abonedir).	(3) Olay yayını ve (4) çoğul bildirim hedefi: hem sensör hem motor, abone tipinden bağımsız olarak yayın yapar. Yeni kanal/abone eklemek mevcut yayıncıyı değiştirmez.

1.3 Reddedilen Alternatifler

ALTERNATİF	NEDEN TERCİH EDİLMEDİ?
Singleton — AkıllıEvIzlemeMotoru 'nu tekil yapmak.	Senaryo "bir akıllı ev" den bahsediyor; ancak <i>test edilebilirlik</i> ve <i>gelecek çok-evli</i> (apartman, site) senaryosu için tekillik bağımlılığı maliyetli. Tekillik bir <i>üretim/yaratım</i> kararı; mevcut tasarım <i>davranışsal</i> sorunları çözer.
Strategy — modları CalismaStratejisi olarak taşımak.	Strategy, davranışı <i>dışarıdan</i> seçilen bir algoritma olarak modeller; biz içine ait <i>durumu</i> modelliyoruz. Özellikle "iki sistemin modu birlikte değişir" kuralı, sistemden ayrı bir strateji nesnesini gerektirmez. State daha doğru kavramsal eşleşmedir.
Pub/Sub mesaj kuyruğu — sensör olayları için.	Senaryo bunu istemez; ek altyapı maliyeti getirir. Observer, <i>aynı süreç içinde</i> ihtiyaç duyulan ayrışmayı sade biçimde sağlar.
Visitor — mod değişikliğinde sistemleri dolaşmak için.	Visitor, <i>sabit nesne hiyerarşisi üzerinde değişken işlemler</i> için uygundur. Burada işlem (mod değiştirme) sabit, hiyerarşi (sistemler) genişleyebilir; tam ters yön. Mediator + State daha uygun.

Mimari kanaat. *State davranışı, Mediator koordinasyonu, Observer ise bilgi yayınına bağımsız eksenler olarak ayırır. Üç örüntü birbirinin kaldıracı: birini çıkartmak diğer ikisinin yükünü artırır ve kodun açılan/kapanan dengesini bozar.*

1.4 Sınıf Diyagramı



Şekil 1.1 · Akıllı Ev Yönetim Sistemi — UML 2.x sınıf diyagramı. Sol katman: kullanıcı ve görüntüleme; orta katman: arabulucu motor ve gözlemci arayüzleri; alt katman: sistem ve sensör hiyerarşileri.

1.5 Sınıf Açıklamaları

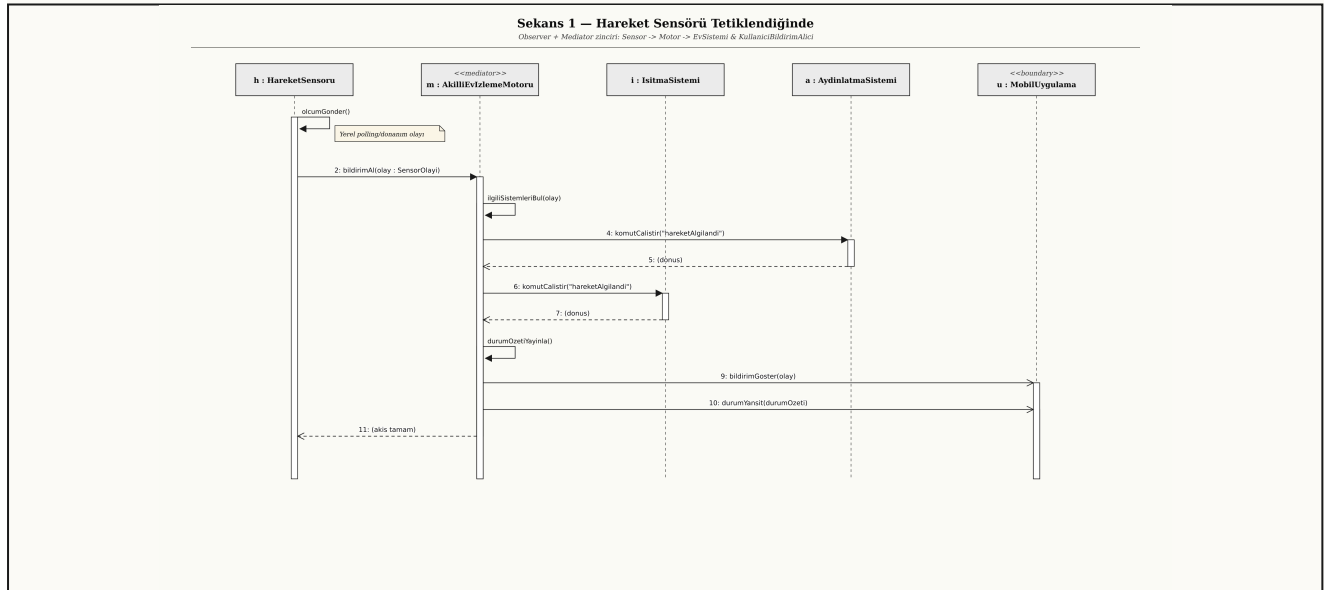
SINIF / ARAYÜZ	STEREOTİP	SORUMLULUK & ROLLER
Kullanici	«entity»	Ev sahibinin kimliği; bir veya birden fazla bildirim alıcıya (mobil/web) sahip olabilir.
KullaniciBildirimAlici	«interface»	Observer arayüzü (kanal-bağımsız). bildirimGoster(olay) ve durumYansit(ozet) .
MobilUygulama . WebUygulamasi	«concrete»	Somut gözlemci; kanal-özel görüntüleme yapar. Kullanıcı komutlarını motora iletir.
DurumOzeti	«value object»	Anlık durum fotoğrafı; değişmez (immutable). Motor üretir, alıcılar tüketir.
AkilliEvIzlemeMotoru	«mediator» + «observer»	Tüm sistem ve sensörleri tanır. Sensör olaylarında gözlemci, kullanıcı için yayıncı. Mod senkronizasyonu, komut yönlendirme ve durum yayını.
EvSistemi	«abstract»	State için bağlam; aktifMod alanı üzerinden davranışını polimorfik seçer. komutCalistir alt sınıflara bırakılır.
AydinlatmaSistemi . IsitmaSistemi	«concrete»	Cihaz-özel davranışı (asgari/maksimum aydınlatma; hedef sıcaklık) uygular.
CalismaModu	«interface»	State arayüzü; uygula(sistem) ile bağlamı yapılandırır.
GuvenliMod . TasarrufluMod	«concrete»	İki politik davranış; maksimum kapasite ile asgari/hedef ayar.
ModTipi	«enumeration»	Kullanıcı arayüzünden gelen mod tercihi için tip-güvenli sabitler.
Sensor	«abstract»	Özne (subject) rolü: attach / detach ve bildir ortak metotları.
KapiSensoru . HareketSensoru	«concrete»	Donanıma özgü ölçüm metotları (olcumGonder); olay tetiklerler.
SensorGozlemcisi	«interface»	Sensör olaylarını alacak nesneler için Observer arayüzü (motor uygular).

SINIF / ARAYÜZ	STEREOTİP	SORUMLULUK & RÖL
SensorOlayi	«value object»	Tip, kaynak sensör, veri ve zaman taşıyan değişmez olay nesnesi.
SensorTipi	«enumeration»	KAPI / HAREKET; mantık kontrolü ve kayıt için.

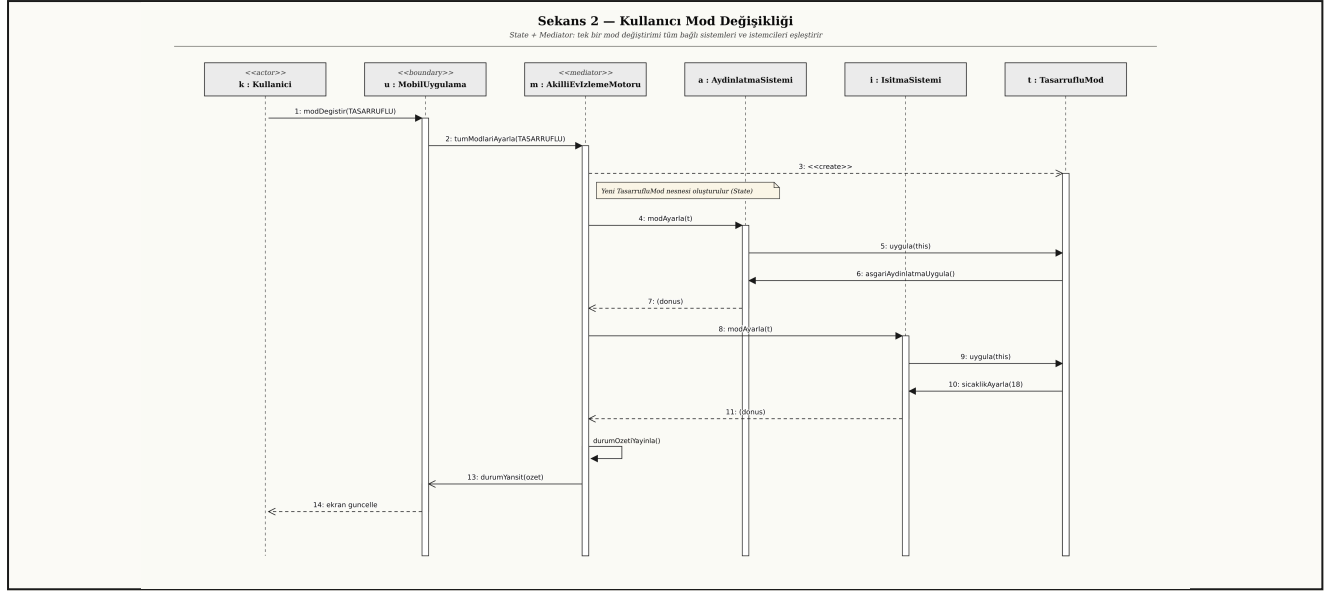
1.6 İlişki Tipleri (UML Notasyonu)

- Composition.** AkıllıEvIzlemeMotoru ♦— EvSistemi (1..*) ve AkıllıEvIzlemeMotoru ♦— Sensor (1..*) : motor olmadan sistem/sensör anlamlı değildir — yaşam döngülerini motor sahiplenir.
- Aggregation.** AkıllıEvIzlemeMotoru ◊— KullaniciBildirimAlici (0..*) : bildirim alıcıları motorun parçası değildir, dış dünyaya aittir; motor onları yalnızca tutar.
- Aggregation.** EvSistemi ◊— CalismaModu (aktif mod): mod nesnesi sistemin yaşam döngüsünden bağımsız oluşturulup atanır.
- Realization.** MobilUygulama / WebUygulamasi —..▷ KullaniciBildirimAlici ; GuvenliMod / TasarrufluMod —..▷ CalismaModu ; AkıllıEvIzlemeMotoru —..▷ SensorGozlemcisi .
- Inheritance.** Sensor ve EvSistemi hiyerarşileri.
- Dependency.** AkıllıEvIzlemeMotoru —..> DurumOzeti («creates»): motor durum özetini oluşturup yayınlıyor; alıcılar bunu yalnızca tüketir.

1.7 Sıra Diyagramları (Çalışma Akışları)



Şekil 1.2 · Hareket sensörü tetiklendiğinde olay akışı. Sensör olayı gözlemci üzerinden motora iletilir; motor ilgili sistemleri uyarır, sonra DurumOzeti ile tüm bildirim alıcılarını eşleştirir.



Şekil 1.3 · Kullanıcının tasarruflu mod talebi. Tek bir **tumModlarıAyarla** çağrısı, mediator aracılığıyla tüm sistemleri ve istemcileri tutarlı biçimde günceller; sistemler birbirini referans almaz.

1.8 Tasarımın Niteliksel Avantajları

- **Açık-Kapalı (OCP).** Yeni mod (UykuModu), yeni sensör (Pencere), yeni bildirim kanalı (E-posta); her durumda *yalnızca yeni bir sınıf* eklenir, mevcut kod değişmez.
- **Tek Sorumluluk (SRP).** Mod davranışı **CalismaModu** hiyerarşisinde; senkronizasyon politikası **AkıllıEvIzlemeMotoru** 'nda; olay taşıması **Sensor** + **SensorGozlemcisi** ikilisinde tutulmuştur.
- **Düşük Bağlılık.** Sistemler birbirini, sensörler de motoru somut bir tip olarak bilmez.
- **Test Edilebilirlik.** Stub gözlemci ve sahte modlar ile motor birim test seviyesinde doğrulanabilir.
- **Liskov Uyumu.** Yeni mod, yeni sistem ve yeni sensör türleri üst sözleşmenin (*contract*) ihlali olmadan eklenir.

SORU 2 · 50 PUAN

SimpleCar E-Satış Sistemi

Strategy, Chain of Responsibility, Command ve Memento Örüntülerinin Birlikte Kullanımı — Coğrafi Genişleme için Abstract Factory

2.1 Senaryonun Çözümlemesi

SimpleCar firması yalnızca e-satış yapan bir araba üreticisidir. Domeni altı bileşene ayrılabilir:

- 1. Algoritma ailesi (ödeme).** Ödeme tek bir *davranıştır*, ancak *banka havalesi*, *paypal*, *soğuk cüzdan* gibi farklı yollarla yapılır; senaryo ayrıca bu listenin *sıklıkla genişleyip daralacağını* özellikle vurgular. Çalışma zamanında seçilebilen, bağımsız olarak değiştirilebilen bir *algoritma ailesi* gerekir.
- 2. Aşamalı ve genişleyebilir kontrol.** Her sipariş *sahtecilik*, *limit* ve *bakiye* kontrollerine tabidir. Birinin olumsuzluğu siparişi iptal eder. Senaryo bu kontrollerin de zamanla *artırılabilceğini* belirtir. Bu, bir *filtre zinciri*: her halka kararı verir, başarılıysa sıradakine devreder; başarısızsa zinciri keser.
- 3. Sıralı işleme adımları.** Tüm kontroller başarılıysa sipariş sırasıyla *fatura düzenleme*, *fatura gönderme* ve *alacaklara kaydetme* adımlarıyla işlenir. Adımlar ileride değişebilir; her biri test edilebilir bağımsız bir iş birimini temsil eder.
- 4. Kısmi başarısızlıkta tutarlılık.** Üç işleme adımı ardışık ve *yan etkilidir* (fatura yaratılır, e-posta gider, defter kaydı atılır). Senaryo, n. adımın hata vermesi halinde sistemi tutarsız bir ara durumda bırakmamayı zımnen gerektirir: yapılan adımlar ters yönde telafi edilebilmelidir.
- 5. Yönetim ve yaşam döngüsü.** Bir orchestrator (use-case) bileşen, siparişi *alır*, *doğrular* ve *işler*. Domain (Siparis) ile servis sorumlulukları ayrık tutulmalıdır.
- 6. Coğrafi genişleme imkânı.** Soruda — “*başka ülkelere dağıtım yapabilirsiniz*” bilgisi verilir. Yani sistem, ülkeye bağlı olarak *kabul edilen ödeme yöntemlerini*, *uygulanan kontrol kurallarını* (KVKK / GDPR, yaptırım listeleri, ihracat lisansı), *ek işleme adımlarını* (gümrük beyannamesi, e-fatura biçimi) ve *para birimini* ülkeye göre değiştirebilmelidir. Bu, tek bir ekstra eksenidir; çekirdek tasarım bunu «Abstract Factory» ile ayrı bir katmana iter (bkz. 2.10).

2.2 Örüntü Seçimleri ve Gerekçeleri

ÖRÜNTÜ	UYGULANAN YER	ÇÖZDÜĞÜ OLGU
Strategy GoF, s.315	OdemeYontemi arayüzü; BankaHavalesi, Paypal, SogukCuzdan ; siparişle 1-1 birikim.	(1) Algoritma ailesi: ödeme ailesi sipariştten bağımsız olarak yer

ÖRÜNTÜ	UYGULANAN YER	ÇÖZDÜĞÜ OLGU
		değiştirebilir; yeni yöntem eklemek mevcut kodu değiştirmez.
Chain of Responsibility <i>GoF, s.223</i>	<code>SiparisKontrolu</code> soyut sınıfı; <code>Sahtecilik</code> , <code>Limit</code> , <code>Bakiye</code> halkaları; <code>setSonraki</code> ile kuruluyor.	(2) Aşamalı kontrol: halka başarılıysa sıradakine devreder, başarısızsa zincir kesilir; yeni kontrol eklemek tek satır konfigürasyondur.
Command + Macro Command <i>GoF, s.233</i>	<code>Komut</code> arayüzü; <code>FaturaDuzenle</code> , <code>FaturaGonder</code> , <code>AlacaklaraKaydet</code> ; <code>SiparisIsleyici</code> sırayla çalıştırır.	(3) Sıralı işleme adımları: her adım kapsüllenmiş; sıra değiştirme, ek adım, geri-alma (undo) gibi ihtiyaçlar için esnek altyapı.
Memento <i>GoF, s.283</i>	<code>SiparisDurumKaydi</code> «memento»; <code>Siparis</code> «originator»; <code>SiparisIsleyici</code> «caretaker» (kayıt yığını).	(4) Kısmi başarısızlıkta tutarlılık: yapılan adımlar ters sırayla <code>geriAl(s, k)</code> ile çevrilir; siparişin iç alanları <code>Komut</code> 'a açılmaz, kapsülleme korunur.
Abstract Factory <i>GoF, s.87</i>	<code>BolgeKonfigurasyonu</code> «abstract factory»; <code>TrKonfigurasyonu</code> , <code>DeKonfigurasyonu</code> somut fabrikalar; ülkeye-uygun ödeme/kontrol/komut takımını <code>SiparisYoneticisi</code> 'ne sağlar (bkz. Şekil 2.3).	(5) Coğrafi genişleme: ülke başına farklı paket gerekiyor; <i>aileler birlikte değişir</i> (yeni ülkenin ödeme + kontrol + komutları tutarlı bir küme oluşturmalıdır). Tek-noktadan kuruluş, çekirdek sınıflara dokunmadan n ülkeye ölçeklenir.

Tasarım notu. GoF, Command bölümü s.237'de "A command can use a memento to maintain the state required to reverse its effect" der. Burada bu kalıp, makro komutun *caretaker* rolünü üstlenmesiyle Saga'nın yerel (in-process) bir sürümüne dönüştürülmüştür: dağıtık koordinasyon olmadan, tek transaksyon sınırı içinde tutarlılık.

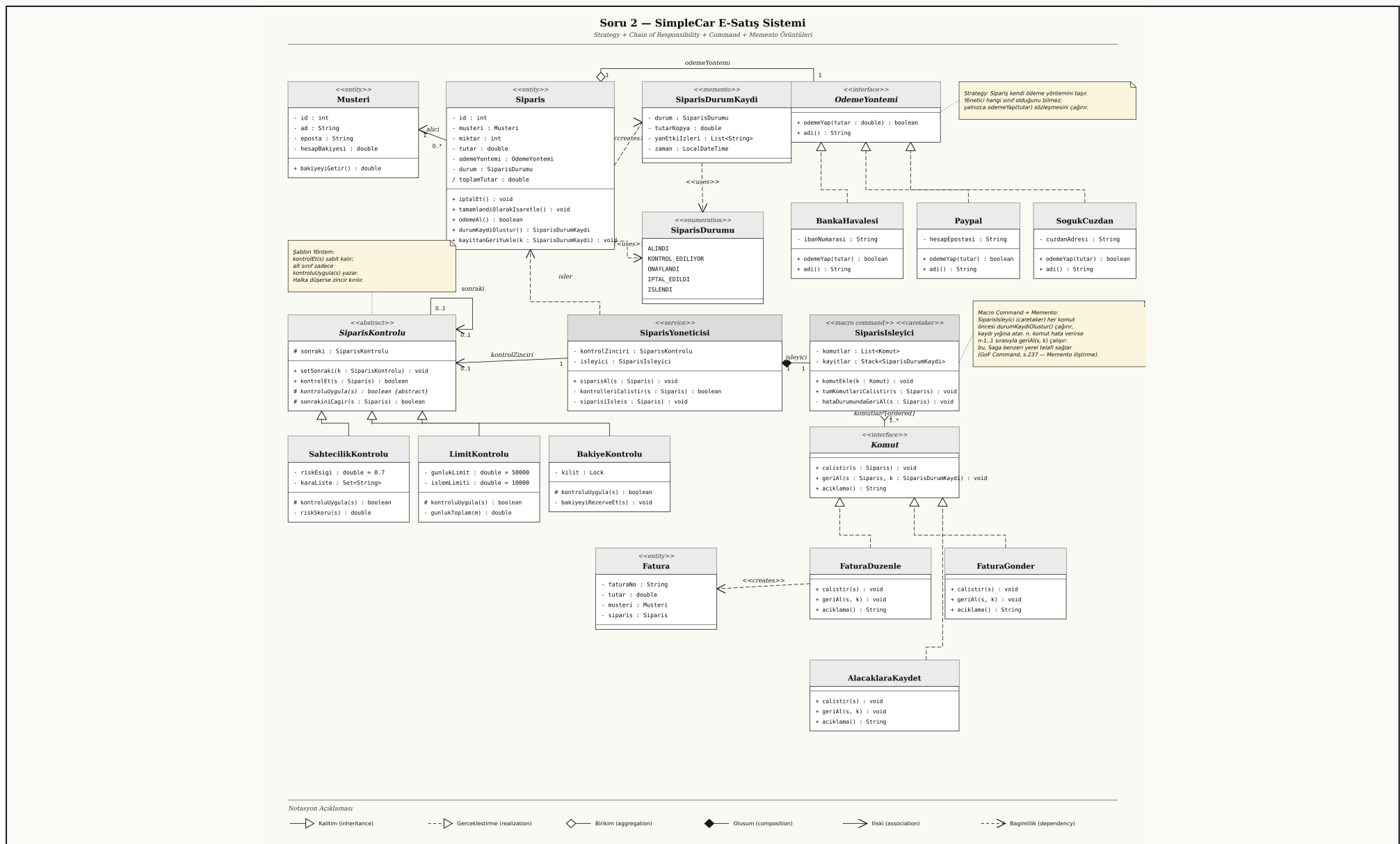
2.3 Reddedilen Alternatifler

ALTERNATİF	NEDEN TERCİH EDİLMEDİ?
Template Method — sipariş işleme akışını soyut sınıfta şablon olarak kurmak.	Şablon, <i>sıra sabit</i> ise iyidir; ancak senaryo "adım sayısı değişebilir" implikasyonunu taşır (yeni adımlar). Komut listesi, çalışma zamanında yeniden düzenlenebilir; şablon ise sınıfların yeniden yazılmasını gerektirir.
Decorator — kontrolleri	Decorator, ana arayüze yeni davranış ekler; ancak burada kontroller <i>kararsal</i> : false dönerse akışı durdurur. Bu, "ekleme" değil "eleme" semantiği; CoR daha doğru.

ALTERNATİF	NEDEN TERCİH EDİLMEDİ?
sarmalayıcı olarak eklemek.	
Observer — sipariş durumu değiştiğinde abonelere haber vermek.	Faydalı olabilirdi; ancak senaryo "bilgilendirme" istemiyor, <i>işlem akışını</i> tarif ediyor. Çözümümüz işlem-yönlüdür; Observer ihtiyaca göre daha sonra eklenebilir, mimariyi engellemez.
State — sipariş durumu için.	<code>SiparisDurumu</code> bir enum yeterli; siparişin davranışı durumla değişmiyor (yalnızca basit gözlemlenebilir alanlar). State aşırı mühendislik olur.

Mimari kanaat. *Strategy "ne ile ödensin", CoR "kabul edilebilir mi", Command "kabul sonrası ne yapılsın" sorularını birbirinden bağımsız cevaplar. Senaryonun her üç değişme eksenini — ödeme yöntemleri, kontrol kuralları, işleme adımları — ayrı bir örüntü ile soyutlanmıştır; biri büyürken diğerleri sabit kalır.*

2.4 Sınıf Diyagramı



Şekil 2.1 · SimpleCar E-Satış — UML 2.x sınıf diyagramı. Sol kanat: CoR (kontroller); orta: orchestrator; sağ üst: Strategy (ödeme yöntemleri); sağ alt: Command (sıralı işleme adımları).

2.5 Sınıf Açıklamaları

SINIF / ARAYÜZ	STEREOTİP	SORUMLULUK & ROLLER
Musteri	«entity»	Siparişi veren kişi; <code>hesapBakiyesi</code> bakiye kontrolünde kullanılır.
Siparis	«entity» / «originator»	Aggregate root; tutar, miktar, seçilen <code>OdemeYontemi</code> , durum bilgilerini taşır. Memento rolünde de <i>originator</i> 'dur: <code>durumKaydiOlustur()</code> ile kendi iç durumunu kapsüllü bir <code>SiparisDurumKaydi</code> olarak dışa verir, <code>kayittanGeriYukle(k)</code> ile geri okur.
SiparisDurumu	«enumeration»	Yaşam döngüsü sabitleri: ALINDI / KONTROL_EDILIYOR / ONAYLANDI / IPTAL_EDILDI / ISLENDI.
SiparisDurumKaydi	«memento»	Siparişin belirli bir andaki <i>kapsüllenmiş</i> kopyası: durum, tutar, yan etki izleri ve zaman damgası. Sadece <code>Siparis</code> oluşturup okur; <code>SiparisIsleyici</code> kayıt yığnında tutar (caretaker). Komutlar bu nesneye <i>opak</i> bakar — <i>narrow interface</i> ilkesi.
OdemeYontemi	«interface»	Strategy sözleşmesi; <code>odemeYap(tutar)</code> imzası.
BankaHavalesi . Paypal . SogukCuzdan	«concrete»	Mevcut ödeme stratejileri; her birinin kanal-özel alanları (IBAN, e-posta, cüzdan adresi) vardır.
SiparisKontrolu	«abstract»	CoR taban sınıfı; <code>kontrolEt</code> şablon metodu zincirleme işini taşır, <code>kontrolUygula</code> alt sınıflara bırakılır.
SahtecilikKontrolu . LimitKontrolu . BakiyeKontrolu	«concrete»	Tek bir kuralı uygulayan halkalar; bağımsız birim test yapılabilir.
Komut	«interface»	Command sözleşmesi; ileri yön <code>calistir(s)</code> , geri yön <code>geriAl(s, k)</code> — <i>k</i> , ilgili adımdan önce alınmış Memento'dur. Bu birleşim, GoF Command, s.237 (<i>Implementation</i> , madde 6) tarafından önerilir.
FaturaDuzenle . FaturaGonder . AlacaklaraKaydet	«concrete»	Senaryoda istenen sıralı üç işleme adımı; her biri kendi yan etkisinden sorumlu. Her komut, <code>geriAl</code> 'da kendi yan etkisini ters yönde işler (faturayı sil, gönderimi iptal et, alacak kaydını sil).

SINIF / ARAYÜZ	STEREOTİP	SORUMLULUK & RÖL
SiparisIsleyici	«macro command» / «caretaker»	Sıralı komut listesi; <code>komutEkle</code> ile genişletilebilir. Aynı zamanda Memento <i>caretaker</i> 'idir: her komut çağrısından önce <code>durumKaydiOlustur()</code> sonucunu <code>kayitlar</code> yığınınına iter; <i>n</i> . komut hata verirse <code>hataDurumundaGeriAl(s)</code> içinde <i>n-1..1</i> sırasıyla <code>geriAl(s, k)</code> çağırır. Kayıtların içine asla <i>bakmaz</i> — sadece taşır (narrow interface).
SiparisYoneticisi	«service»	Use-case orchestratoru; kontrol zincirini ve işleyiciyi koordine eder. <i>Açık</i> : burada Facade kalıtsal bir izi vardır — çağırana tek bir <code>siparisAl(s)</code> arayüzü sunulur.
Fatura	«entity»	Fatura komutları arasında paylaşılan veri; sipariş ile <i>1-1</i> ilişkili.

2.6 İlişki Tipleri (UML Notasyonu)

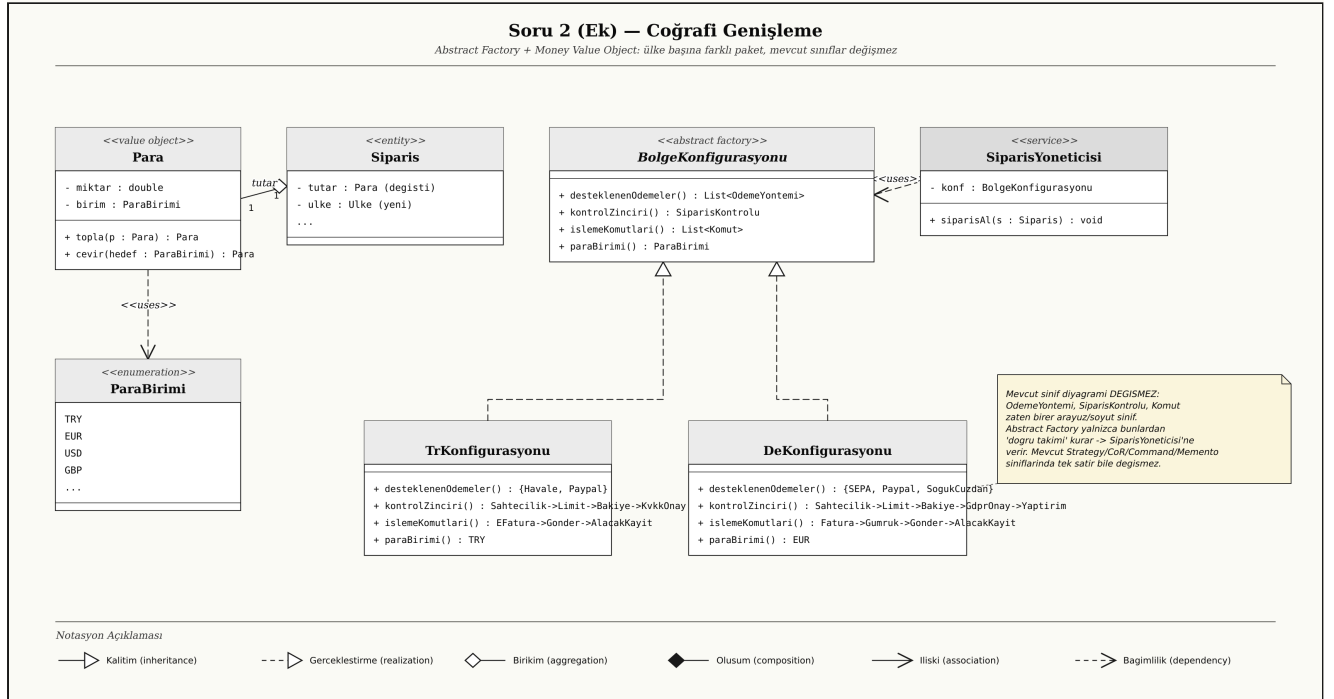
- *Aggregation*. `Siparis` ◊— `OdemeYontemi` (1..1) : ödeme yöntemi siparişe atanır, fakat onun parçası değildir.
- *Aggregation*. `SiparisIsleyici` ◊— `Komut` (1..*, {ordered}) : komut listesi işleyiciden bağımsız olarak da var olabilir; UML kısıtı {ordered} sıralı koleksiyonu belirtir.
- *Composition*. `SiparisYoneticisi` ◆— `SiparisIsleyici` : yönetici, işleyiciyi kendisi oluşturur ve sahiplenir.
- *Self-association*. `SiparisKontrol` — sonraki: 0..1 : zincirleme bağıntısı, halkanın kendisine.
- *Realization*. `BankaHavalesi/Paypal/SogukCuzdan` —..> `OdemeYontemi` ;
`FaturaDuzenle/FaturaGonder/AlacaklaraKaydet` —..> `Komut` .
- *Inheritance*. Üç kontrol halkası `SiparisKontrol` 'ndan türer.
- *Dependency*. `SiparisYoneticisi` —..> `Siparis` (etiket: *isler*); `FaturaDuzenle` —..> `Fatura` («creates»); `Siparis` —..> `SiparisDurumKaydi` («creates»); `Siparis` —..> `SiparisDurumu` ve `SiparisDurumKaydi` —..> `SiparisDurumu` («uses» — her iki sınıf da `durum` alanını bu enumerasyondan tipler).

2.10 Coğrafi Genişleme (Multi-Region Dağıtım)

Soruda — *"başka ülkelere dağıtım yapabilirsiniz"* imkânı verildiğinden, mevcut tasarımın bu ekseninde ne kadar değişeceğini gösteriyorum. Cevap kısa: **çekirdek sınıf diyagramı değişmez**; üzerine iki küçük katman eklenir.

- Money (Para) value object.** Siparis.tutar : double artık yetmez; tutar bir miktar ile bir *para* biriminden oluşur. Bu, bir tip değişikliğidir, mimari değişiklik değildir. Aritmetik kuralları Para içinde toplanır; karşılaştırma/dönüşüm tek bir yerden geçer.
- BolgeKonfigurasyonu «abstract factory».** Ülkeye göre değişen üçlüyü — kabul edilen *ödeme yöntemleri*, çalıştırılacak *kontrol zinciri*, sırasıyla yürütülecek *komut listesi* — tek bir noktadan kuran fabrika. Her ülke için somut bir konfigürasyon sınıfı yazılır (TrKonfigurasyonu , DeKonfigurasyonu vb.).
SiparisYoneticisi doğrudan bu fabrikadan istediği paketi alır; mevcut Strategy/CoR/Command sınıflarını ne tanır ne değiştirir.

Aşağıdaki ek diyagram bu iki katmanı, *asıl diyagramı kalabalıklaştırmamak için ayrı* gösterir.



Şekil 2.3 · Coğrafi genişleme katmanı. **BolgeKonfigurasyonu**, mevcut arayüzlerden (**OdemeYontemi**, **SiparisKontrolu**, **Komut**) doğru takımı kurar. Mevcut sınıf diyagramı (Şekil 2.1) hiç değişmez.

2.10.1 Etkilenen ve Etkilenmeyen Kısımlar

KISIM	ETKİ
Strategy ailesi (OdemeYontemi + somut sınıflar)	Değişmez. Yeni ülkeye yeni ödeme gelirse Strategy ailesine yeni bir sınıf eklenir.

KISIM	ETKİ
CoR halkaları (SiparisKontrolü + somut halkalar)	Değişmez. Ülkeye-özgü halkalar (KVKK/GDPR onayı, yaptırım listesi, ihracat lisansı) yeni somut halkalar olarak eklenir.
Command + Macro Command + Memento	Değişmez. Ülkeye-özgü adımlar (gümrük beyannamesi, e-fatura UBL üretimi, döviz dönüşümü) yeni Komut uygulamaları olarak eklenir; geriAl mantığı her komutun kendi sorumluluğundadır.
Siparis	İki alan değişir/eklenir: tutar : double → tutar : Para ve ülke : Ülke . İmzaların geri kalanı korunur.
SiparisYoneticisi	Bir alan eklenir: konf : BölgeKonfigurasyonu ; ilk açılışta hangi ülke için çalışıldığına göre fabrikadan paketi alır.
Lojistik / sevkiyat (kargo, depo, taşıyıcı, gümrük takibi)	Yeni bağlam (bounded context). Bu, e-satış agregasının içine sıkıştırılmaz; mesaj/event ile haberleşen ayrı bir alt-sistem olur. Mevcut tasarımı bozmaz, <i>yanına</i> gelir.
Yerelleştirme (dil, fatura biçimi, tarih/sayı)	Tek bir FaturaBicimleyici Strategy'si; FaturaDuzenle komutuna enjekte edilir. Yeni sınıf değil, mevcut komutun parametreleşmesi.

2.10.2 Tek Cümlelik Karar

Mevcut çekirdek tasarımın tüm değişen eksenleri (*ödeme, kontrol, işleme adımı*) zaten arayüz/soyut sınıf seviyesinde soyutlanmış olduğundan, ülke bazında dağıtım *yeni varyant eklemek* meselesidir; bunları toplayan tek-noktalı bir *Abstract Factory* ile sistem, çekirdeği yeniden yazmadan **n adet ülkeye** ölçeklenir. Asıl yeni iş, lojistik bağlamı ve **Para** tipidir; bu da mevcut sınıflara dokunmadan yapılır.

2.11 Sonuç

Önerilen tasarım; siparişin *alınması, doğrulanması ve işlenmesi* iç akışlarını birbirinden ayrı, örüntü-tabanlı kapsüllere yerleştirir. Senaryonun en sık değişeceği belirtilen iki eksen — *ödeme yöntemleri* ve *kontrol kuralları* — mevcut kodu değiştirmeden büyür. Sıralı işleme; gözlemlenebilir, sıra/komut listesi değiştirilebilir ve test edilebilir bir komut akışı ile gerçekleştirilir; ardışık yan etkilerin ortasında oluşacak hatalar Memento + caretaker yığını üzerinden *tutarlı bir önceki ana* geri sarılır. Aynı çekirdek, ülke bazlı dağıtıma da bir *Abstract Factory* katmanı ile genişler; çekirdek sınıflar değişmeden farklı ülke paketleri kurulur. Böylece hem sınav kurallarında istenen *örüntü adı açıkça belirtilmiş* ve *ilişki tipi ile işaretlemeleri doğru kullanılmış*; hem de senaryodaki nitelikler ve metotlar sınıflar içinde konumlandırılmıştır.